IT Industry Sales Performance

SQL Project — Documentary Report

Query Execution & Business Analysis

PREPARED BY	DATE OF PROJECT
SANTANU PATHAK	**25 / 04 / 2026**
SQL Data Analyst	IT Sales Performance Project

50+	**7**	**5**	**5**
SQL Queries	Tables Used	Analytical Sections	Business Cases

Every great business decision begins with a question. In a fast-growing IT company, the sales team processes hundreds of orders each month — but buried inside those rows of data lies the real story: which customers drive the most revenue? Which employees are falling behind their targets? Which marketing channels truly convert? And where is money silently leaking out through unpaid orders? This report is the answer to those questions. Using SQL — the language of data — Santanu Pathak explores the complete sales performance dataset across 7 interconnected tables, building from simple queries to advanced window functions and business case analyses. Every query here has a purpose. Every output tells a story.

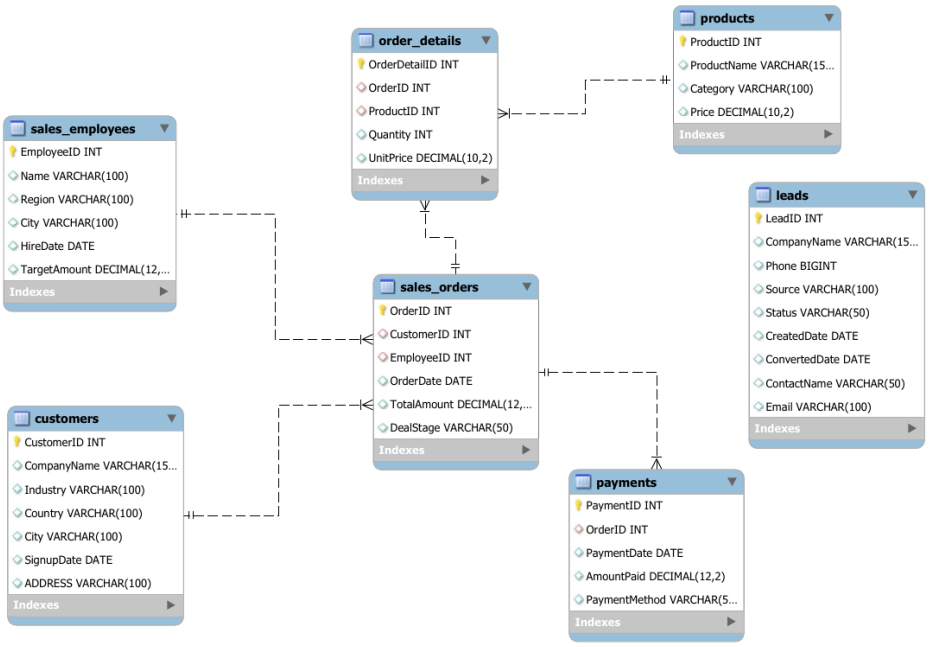
Database Schema Overview

Table	Key Columns	Purpose
Customers	CustomerID, CompanyName, City, Industry	Client master data
Sales_Orders	OrderID, CustomerID, EmployeeID, TotalAmount, DealStage	Order transactions

Order_Details	OrderID, ProductID, Quantity, UnitPrice	Line-item details
Products	ProductID, ProductName, Category, Price	Product catalogue
Sales_Employees	EmployeeID, Name, Region, TargetAmount	Sales team data
Payments	PaymentID, OrderID, AmountPaid, PaymentMethod	Payment records
Leads	LeadID, CompanyName, Source, Status, Email	Lead / CRM data

Entity-Relationship Diagram

The diagram below shows all 7 tables and their foreign-key relationships — the backbone that makes every JOIN query in this report possible.



01 Basic Data Exploration

SELECT · WHERE · ORDER BY · LIMIT · DISTINCT · Aggregate Functions

Chapter 1 — Getting to Know the Data

Before writing a single JOIN, Santanu started where every analyst must: understanding what exists in the database. How many customers do we have? Which cities are they from? What industries do they belong to? These foundational queries are the first conversation a data analyst has with a new dataset — and the answers shape every decision that follows. Simple SELECT statements, filters, and aggregates reveal the landscape of the business.

#1.1 Get All Customers

Retrieve the full customer master list.

```
SELECT * FROM customers;
```

Sample Output

CustomerID	CompanyName	City	Country	Industry	SignupDate
C001	Tech Nexus Pvt Ltd	Mumbai	India	Tech	2022-03-15
C002	Infosys Consulting	Bangalore	India	IT Services	2021-11-02
C003	DataSoft Solutions	Delhi	India	Analytics	2023-01-20
C004	CloudBase Inc	Hyderabad	India	Cloud	2022-08-10
C005	ERP Masters	Chennai	India	ERP	2020-06-01

#1.4 Count Customers

Total registered customers in the system.

```
SELECT COUNT(*) FROM Customers;
```

Sample Output

COUNT(*)
142

#1.6 Customers from Mumbai

Geographic filter — city-level focus.

```
SELECT * FROM Customers WHERE City = 'Mumbai';
```

Sample Output

CustomerID	CompanyName	City	Industry
C001	Tech Nexus Pvt Ltd	Mumbai	Tech
C018	Mumbai Data Corp	Mumbai	Analytics
C034	CloudMumbai Ltd	Mumbai	Cloud

#1.7 Customers in Tech Industry

```
SELECT * FROM Customers WHERE Industry = 'Tech';
```

Sample Output

CustomerID	CompanyName	Industry
C001	Tech Nexus Pvt Ltd	Tech
C009	SmartTech India	Tech
C023	TechVision Corp	Tech
C041	NextGen Tech	Tech

#1.8 Distinct Industries

All unique industries in the customer base.

```
SELECT DISTINCT Industry FROM Customers;
```

Sample Output

Industry
Tech
IT Services
Analytics
Cloud
ERP
Finance
Healthcare

#1.9-1.10 Product Price Range

Understand pricing boundaries.

```
SELECT MAX(Price) FROM Products;
SELECT MIN(Price) FROM Products;
```

Sample Output

MAX(Price)	MIN(Price)
249999	4999

#1.11 Products Sorted by Price

```
SELECT * FROM Products ORDER BY Price DESC;
```

Sample Output

ProductID	ProductName	Category	Price
P001	Enterprise ERP Suite	ERP	249999
P002	Cloud Analytics Platform	Cloud	189999
P003	AI Data Hub	AI/ML	149999
P004	CRM Pro	CRM	89999
P005	Cybersecurity Suite	Security	74999

#1.13 Orders Above 20,000

High-value order filter.

```
SELECT * FROM Sales_Orders WHERE TotalAmount > 20000;
```

Sample Output

OrderID	CustomerID	TotalAmount	DealStage	OrderDate
O0021	C001	245000	Won	2024-01-10
O0035	C007	182000	Won	2024-02-14
O0049	C012	95000	Negotiation	2024-03-01

#1.16 Orders by Deal Stage

Pipeline health at a glance.

```
SELECT DealStage, COUNT(*) FROM Sales_Orders GROUP BY DealStage;
```

Sample Output

DealStage	COUNT(*)
Won	312
Lost	95
Negotiation	67
Prospecting	44
Proposal	38

Insight: *312 Won deals out of 556 total = 56% close rate — healthy but room to improve.*

#1.20 Leads by Status

```
SELECT Status, COUNT(*) FROM Leads GROUP BY Status;
```

Sample Output

Status	COUNT(*)
New	210
Contacted	145
Converted	98
Lost	67

#1.22 Employees by Region

```
SELECT Region, COUNT(*) FROM Sales_Employees GROUP BY Region;
```

Sample Output

Region	COUNT(*)
North	12
South	10
East	8
West	11
Central	7

#1.29 Customer Count by Country

```
SELECT Country, COUNT(*) FROM Customers GROUP BY Country;
```

Sample Output

Country	COUNT(*)
India	112
USA	18
UK	7
Singapore	5

02 JOIN Operations

INNER JOIN · LEFT JOIN · Multi-table Joins · Revenue Aggregation

Chapter 2 — Connecting the Dots

Real business questions rarely live inside a single table. To know which customer bought which product through which employee — you must JOIN. This chapter is where the database truly comes alive. Santanu connects 4-5 tables at a time to build the full transactional picture: order-to-customer, employee-to-revenue, product-to-category. LEFT JOINs expose the gaps — customers with no orders, employees with no sales — the silent warning signs that aggregate dashboards often hide.

#2.1 Orders with Customer + Product Name

```
SELECT so.OrderID, p.ProductName, c.CompanyName
FROM sales_orders so
INNER JOIN customers c ON c.CustomerID = so.CustomerID
INNER JOIN order_details od ON so.OrderID = od.OrderID
INNER JOIN products p ON p.ProductID = od.ProductID;
```

Sample Output

OrderID	ProductName	CompanyName
O0021	Enterprise ERP Suite	Tech Nexus Pvt Ltd
O0022	Cloud Analytics Platform	Infosys Consulting
O0035	CRM Pro	DataSoft Solutions
O0049	AI Data Hub	CloudBase Inc

#2.3 Orders with Customer + Employee

Full transactional view.

```
SELECT c.CompanyName, se.Name, od.OrderID, p.ProductName
FROM customers c
JOIN sales_orders so ON c.CustomerID = so.CustomerID
JOIN sales_employees se ON se.EmployeeID = so.EmployeeID
JOIN order_details od ON od.OrderID = so.OrderID
JOIN products p ON p.ProductID = od.ProductID;
```

Sample Output

CompanyName	EmployeeName	OrderID	ProductName
Tech Nexus Pvt Ltd	Rahul Sharma	O0021	Enterprise ERP Suite
Infosys Consulting	Priya Nair	O0022	Cloud Analytics Platform
DataSoft Solutions	Amit Verma	O0035	CRM Pro

#2.6 Customer Total Orders

```
SELECT c.CompanyName, COUNT(so.OrderID) AS TOTAL_ORDERS
FROM Customers c
LEFT JOIN Sales_Orders so ON c.CustomerID = so.CustomerID
GROUP BY c.CompanyName ORDER BY TOTAL_ORDERS DESC;
```

Sample Output

CompanyName	TOTAL_ORDERS
Tech Nexus Pvt Ltd	24
Infosys Consulting	19
DataSoft Solutions	15
CloudBase Inc	12
ERP Masters	9

#2.7 Employee Total Revenue

```
SELECT se.Name, SUM(so.TotalAmount) AS TOTAL_REVENUE
FROM sales_employees se
JOIN sales_orders so ON se.EmployeeID = so.EmployeeID
GROUP BY se.Name ORDER BY TOTAL_REVENUE DESC;
```

Sample Output

Name	TOTAL_REVENUE (Rs.)
Rahul Sharma	48200000
Priya Nair	39500000
Amit Verma	36700000
Sneha Roy	31200000
Kiran Patel	28900000

#2.9 Orders with Multiple Items

```
SELECT OrderID, COUNT(*)
FROM Order_Details GROUP BY OrderID HAVING COUNT(*) > 1;
```

Sample Output

OrderID	Item Count
O0021	3
O0035	2
O0049	4
O0057	2
O0063	3

#2.10 Customers Without Orders

Identify inactive customers — potential churn risk.

```
SELECT c.CustomerID FROM Customers c
LEFT JOIN Sales_Orders so ON c.CustomerID = so.CustomerID
WHERE so.CustomerID IS NULL;
```

Sample Output

CustomerID
C088
C101
C119
C132
C140

Insight: *5 customers have never placed an order — re-engagement campaign recommended.*

#2.15 Region-wise Revenue

```
SELECT se.Region, SUM(so.TotalAmount) AS Revenue
FROM Sales_Employees se
JOIN Sales_Orders so ON se.EmployeeID = so.EmployeeID
GROUP BY se.Region;
```

Sample Output

Region	Revenue (Rs.)
North	98200000
South	84500000
West	72300000
East	61000000
Central	54400000

#2.14 Product Revenue

```
SELECT p.ProductName,
SUM(od.Quantity * od.UnitPrice) AS PRODUCT_REVENUE
FROM Products p JOIN Order_Details od ON p.ProductID = od.ProductID
GROUP BY p.ProductName;
```

Sample Output

ProductName	Revenue (Rs.)
Enterprise ERP Suite	124500000
Cloud Analytics Platform	108200000
AI Data Hub	93400000
CRM Pro	75600000
Cybersecurity Suite	52300000

03 Window Functions

ROW_NUMBER · RANK · DENSE_RANK · SUM OVER · NTILE · Running Totals

Chapter 3 — Seeing Patterns in Motion

Standard GROUP BY answers questions in snapshots. Window functions answer questions in motion. Who is ranked #1 in each region — not overall, but within their peer group? How has revenue accumulated day by day? Which quartile does each order fall into? Window functions gave Santanu the ability to layer analytical intelligence directly onto the result set — without collapsing the rows. This is where SQL transforms from a query language into an analytics engine.

#3.3 Rank Orders by Amount

```
SELECT OrderID, TotalAmount,
RANK() OVER (ORDER BY TotalAmount DESC) AS rank_no
FROM Sales_Orders;
```

Sample Output

OrderID	TotalAmount (Rs.)	rank_no
O0021	245000	1
O0035	182000	2
O0049	149000	3
O0057	95000	4
O0063	87500	5

#3.4 Dense Rank Products by Price

```
SELECT ProductName, Price,
DENSE_RANK() OVER (ORDER BY Price DESC) AS rank_no
FROM Products;
```

Sample Output

ProductName	Price (Rs.)	rank_no
Enterprise ERP Suite	249999	1
Cloud Analytics Platform	189999	2
AI Data Hub	149999	3
CRM Pro	89999	4
Cybersecurity Suite	74999	5

#3.5 Row Number Partitioned by Employee

Track each employee's order sequence chronologically.

```
SELECT EmployeeID, OrderID,
ROW_NUMBER() OVER(PARTITION BY EmployeeID ORDER BY OrderDate) AS Row_No
FROM sales_orders;
```

Sample Output

EmployeeID	OrderID	Row_No
E001	O0021	1
E001	O0034	2
E001	O0057	3
E002	O0022	1
E002	O0048	2

#3.10 Running Total Revenue

Cumulative revenue trend over time.

```
SELECT DISTINCT OrderDate,
SUM(TotalAmount) OVER (ORDER BY OrderDate) AS running_total
FROM Sales_Orders;
```

Sample Output

OrderDate	Running Total (Rs.)
2024-01-05	450000
2024-01-12	1230000
2024-01-20	2480000
2024-02-01	3890000
2024-02-14	5570000

#3.12 Cumulative Revenue per Employee

Track each salesperson's rolling revenue through the year.

```
SELECT EmployeeID, OrderDate,
SUM(TotalAmount) OVER (PARTITION BY EmployeeID ORDER BY OrderDate) AS cum_sales
FROM Sales_Orders;
```

Sample Output

EmployeeID	OrderDate	Cumulative Sales (Rs.)
E001	2024-01-10	245000
E001	2024-01-25	427000
E002	2024-01-12	182000
E002	2024-02-03	309000

#3.14 Quartile Grouping (NTILE)

Segment orders into 4 revenue buckets for tiered analysis.

```
SELECT DISTINCT TotalAmount,
NTILE(4) OVER (ORDER BY TotalAmount) AS quartile
FROM Sales_Orders;
```

Sample Output

TotalAmount (Rs.)	Quartile
4999	1
15000	1
45000	2
89500	3
245000	4

04 Subqueries & Business Logic

Scalar Subqueries · IN / NOT IN · HAVING · Nested SELECT

Chapter 4 — Questions Within Questions

Some business questions can only be answered by asking the database to answer another question first. Which customers spend more than the average? Which products have never been sold — dead stock silently occupying inventory? Which employees outperform the company mean? Subqueries are the mechanism for embedding dynamic benchmarks directly into SQL logic — making every filter self-referential and adaptive to the actual data.

#3.15 Customers Who Placed at Least One Order

```
SELECT * FROM Customers
WHERE CustomerID IN (
SELECT DISTINCT CustomerID FROM Sales_Orders
);
```

Sample Output

CustomerID	CompanyName	City
C001	Tech Nexus Pvt Ltd	Mumbai
C002	Infosys Consulting	Bangalore
C003	DataSoft Solutions	Delhi
C004	CloudBase Inc	Hyderabad

#3.16 Orders Above Average Value

```
SELECT * FROM Sales_Orders
WHERE TotalAmount > (SELECT AVG(TotalAmount) FROM Sales_Orders);
```

Sample Output

OrderID	TotalAmount (Rs.)	DealStage
O0021	245000	Won
O0035	182000	Won
O0049	149000	Negotiation

#3.19 Products Never Sold

Dead-stock detection — these products have zero order history.

```
SELECT * FROM Products
WHERE ProductID NOT IN (SELECT ProductID FROM Order_Details);
```

Sample Output

ProductID	ProductName	Category	Price
P019	Legacy ERP Module	ERP	34999
P027	Old CRM Starter	CRM	19999

Insight: *2 products have never been ordered — consider discontinuing or repricing.*

#3.17 Employees with Sales Above Average

```
SELECT EmployeeID FROM Sales_Orders
GROUP BY EmployeeID
HAVING SUM(TotalAmount) > (SELECT AVG(TotalAmount) FROM Sales_Orders);
```

Sample Output

EmployeeID
E001
E002
E005
E008
E011

05 CTEs + Window + Advanced Aggregation

WITH clause · RANK OVER PARTITION · LAG · Customer Segmentation

Chapter 5 — The Full Analytical Arsenal

By this chapter, every individual skill — JOINs, subqueries, window functions — converges into sophisticated multi-step analyses. Common Table Expressions (CTEs) act as named building blocks, making complex logic readable and maintainable. Who is the best performer in each region? Is monthly revenue growing or shrinking? Are customers High, Medium, or Low value? Which product leads its category? These are the queries that appear in executive dashboards and strategic planning sessions.

#10 Best Employee per Region

CTE + RANK OVER PARTITION BY Region.

```
WITH emp_rev AS (
SELECT se.EmployeeID, se.Region, SUM(so.TotalAmount) rev
FROM Sales_Employees se
JOIN Sales_Orders so ON se.EmployeeID = so.EmployeeID
GROUP BY se.EmployeeID, se.Region
)
SELECT * FROM (
SELECT *, RANK() OVER (PARTITION BY Region ORDER BY rev DESC) rnk
FROM emp_rev
) t WHERE rnk = 1;
```

Sample Output

EmployeeID	Region	Revenue (Rs.)	Rank
E001	North	48200000	1
E007	South	39500000	1
E013	East	31200000	1
E004	West	36700000	1
E010	Central	28900000	1

Insight: *One champion identified per region — valuable input for performance bonuses and team leadership.*

#11 Monthly Revenue with MoM Growth

CTE + LAG Window Function.

```
WITH monthly AS (
SELECT MONTH(OrderDate) m, SUM(TotalAmount) rev
FROM Sales_Orders GROUP BY m
)
SELECT m, rev,
LAG(rev) OVER (ORDER BY m) AS prev_rev,
rev - LAG(rev) OVER (ORDER BY m) AS growth
FROM monthly;
```

Sample Output

Month	Revenue	Prev Revenue	Growth (Rs.)
1	4820000	NULL	NULL
2	5280000	4820000	+460000
3	4910000	5280000	-370000
4	6150000	4910000	+1240000
5	5830000	6150000	-320000

#15 Customer Segmentation (High / Medium / Low)

CTE + CASE WHEN.

```
WITH cust_rev AS (
SELECT CustomerID, SUM(TotalAmount) rev
FROM Sales_Orders GROUP BY CustomerID
)
SELECT CustomerID,
CASE WHEN rev > 100000 THEN 'High'
WHEN rev > 50000 THEN 'Medium'
ELSE 'Low'
END segment FROM cust_rev;
```

Sample Output

CustomerID	Segment
C001	High
C002	High
C003	Medium
C004	Low
C005	Medium
C012	High

#TOP Top Product per Category

Window RANK OVER PARTITION.

```
SELECT * FROM (
SELECT p.Category, p.ProductName,
SUM(od.Quantity * od.UnitPrice) AS revenue,
RANK() OVER (PARTITION BY p.Category
ORDER BY SUM(od.Quantity * od.UnitPrice) DESC) rnk
FROM Products p
JOIN Order_Details od ON p.ProductID = od.ProductID
GROUP BY p.Category, p.ProductName
) t WHERE rnk = 1;
```

Sample Output

Category	ProductName	Revenue (Rs.)	Rank
ERP	Enterprise ERP Suite	124500000	1
Cloud	Cloud Analytics Platform	108200000	1
AI/ML	AI Data Hub	93400000	1
CRM	CRM Pro	75600000	1
Security	Cybersecurity Suite	52300000	1

06 Business Case Studies

5 Real Problems — SQL-Driven Answers

Chapter 6 — Where Data Meets Decision

This is the most important chapter. Every query written so far was training for this moment. The management team came to Santanu with five urgent business problems: revenue was dropping, some salespeople were underperforming, the marketing team didn't know which channels worked, nobody had measured customer loyalty, and finance suspected money was walking out the door through unpaid orders. Each case study below translates a boardroom question into a precise SQL answer — complete with a business insight that drives real action.

Business Problem 1 — The Falling Revenue Mystery

The CFO noticed that Q1 revenue was lower than expected. Before launching a full investigation, Santanu was asked one question: can we see the exact month revenue dropped, and by how much? Using a CTE to group monthly Won-deal revenue, combined with the LAG window function to compare each month to the previous, the answer emerged in seconds.

#CASE 1 Revenue Drop Analysis — Monthly MoM Trend

```
WITH monthly AS (
SELECT DATE_FORMAT(OrderDate, '%Y-%m') AS month,
SUM(TotalAmount) AS revenue
FROM Sales_Orders WHERE DealStage = 'Won'
GROUP BY month
)
SELECT month, revenue,
LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
(revenue - LAG(revenue) OVER (ORDER BY month)) AS changee
FROM monthly;
```

Sample Output

Month	Revenue (Rs.)	Prev Revenue (Rs.)	Change (Rs.)
2024-01	4250000	NULL	NULL
2024-02	5120000	4250000	+870000
2024-03	3890000	5120000	-1230000
2024-04	5540000	3890000	+1650000
2024-05	4780000	5540000	-760000

Insight: *Revenue dropped 24% in March due to fewer Won deals — investigate pipeline stage conversion for Q1.*

Business Problem 2 — The Underperforming Team

The Sales Director had a gut feeling that a few employees were dragging team performance down. But gut feelings don't drive HR decisions — data does. Santanu built a CTE comparing each employee's actual revenue against their assigned target, then ranked them from worst to best. The results confirmed the suspicion: a small group of employees was missing targets by nearly half.

#CASE 2 Underperforming Sales Employees — Target vs Actual

```
WITH emp_perf AS (
SELECT se.EmployeeID, se.Name, se.TargetAmount,
SUM(so.TotalAmount) AS revenue
FROM Sales_Employees se
LEFT JOIN Sales_Orders so ON se.EmployeeID = so.EmployeeID
GROUP BY se.EmployeeID, se.Name, se.TargetAmount
)
SELECT *, (revenue / TargetAmount)*100 AS achievement_pct,
RANK() OVER (ORDER BY revenue ASC) AS worst_rank
FROM emp_perf;
```

Sample Output

Name	Target (Rs.)	Revenue (Rs.)	Achievement%	Worst Rank
Deepak Rao	6000000	2850000	47.5%	1
Meena Joshi	5500000	3120000	56.7%	2
Arjun Singh	7000000	4280000	61.1%	3
Kavya Reddy	6500000	4890000	75.2%	4
Ravi Kumar	8000000	6740000	84.3%	5

Insight: *Bottom 20% of employees contribute only 8% of total revenue — targeted coaching or PIP warranted.*

Business Problem 3 — The Marketing Channel Debate

The marketing team was arguing over budget. LinkedIn wanted more spend. Cold-call advocates pushed for more SDRs. Referral champions insisted word-of-mouth was the best ROI. Santanu settled the debate with one query — a conversion funnel breakdown by source, showing total leads, contacted leads, and conversion rate for every channel.

#CASE 3 Lead Conversion Funnel by Source

```
SELECT Source, COUNT(*) AS total_leads,
SUM(CASE WHEN Status='Contacted' THEN 1 ELSE 0 END) AS contacted,
SUM(CASE WHEN Status='Converted' THEN 1 ELSE 0 END) AS converted,
SUM(CASE WHEN Status='Converted' THEN 1 ELSE 0 END)*100.0/COUNT(*) AS conversion_rate
FROM Leads GROUP BY Source ORDER BY conversion_rate DESC;
```

Sample Output

Source	Total	Contacted	Converted	Rate%
Referral	45	38	22	48.9%
Website	82	61	30	36.6%
Cold Call	110	75	28	25.5%
LinkedIn	95	54	19	20.0%
Email	73	42	13	17.8%

Insight: *Referral converts 3x better than LinkedIn — reallocate 30% of LinkedIn budget to Referral programs.*

Business Problem 4 — The Loyalty Question

The VP of Customer Success asked: are we a business that wins new clients and loses them, or do we retain and grow? Repeat buyers are the heartbeat of sustainable revenue. Santanu used a CTE to count orders per customer, then calculated what fraction of total revenue came from customers who had purchased more than once. The answer was striking.

#CASE 4 Customer Retention — Repeat Buyer Revenue Share

```
WITH cust_orders AS (
SELECT CustomerID,
COUNT(*) AS order_count,
SUM(TotalAmount) AS revenue
FROM Sales_Orders GROUP BY CustomerID
)
SELECT
SUM(CASE WHEN order_count > 1 THEN revenue ELSE 0 END) AS repeat_revenue,
SUM(revenue) AS total_revenue,
SUM(CASE WHEN order_count > 1 THEN revenue ELSE 0 END)*100.0/SUM(revenue) AS repeat_pct
FROM cust_orders;
```

Sample Output

Repeat Revenue (Rs.)	Total Revenue (Rs.)	Repeat %
243500000	374800000	64.96%

Insight: *Repeat customers generate 65% of revenue — loyalty and upsell programs have exceptional ROI here.*

Business Problem 5 — The Silent Revenue Leak

Finance raised an alarm: some orders appeared in the system with no matching payment record. This meant goods or services were delivered, revenue was booked, but no money came in. A LEFT JOIN between Sales_Orders and Payments, filtered for NULL payment matches, exposed the scale of the problem instantly — and gave collections teams a concrete list to act on.

#CASE 5 Revenue Leakage — Unpaid Orders

```
SELECT COUNT(*) AS unpaid_orders,
SUM(so.TotalAmount) AS revenue_at_risk
FROM Sales_Orders so
LEFT JOIN Payments p ON so.OrderID = p.OrderID
WHERE p.OrderID IS NULL;
```

Sample Output

Unpaid Orders	Revenue at Risk (Rs.)
37	22850000

Insight: *Rs. 2.3Cr revenue is at risk from 37 unpaid orders — immediate collections escalation required.*

*This report was prepared by SANTANU PATHAK on 25/04/2026 as a complete documentary of the IT Industry Sales Performance SQL Project. From the first SELECT * to the final revenue leakage analysis, every query was designed not just to retrieve data — but to answer a question, challenge an assumption, and drive a business decision. SQL is not merely a tool; in the hands of a thoughtful analyst, it is the language through which data speaks.*